

SESSION 15

Django Authentication (Advanced)

Topics Covered: Login required decorator, Role-based access (doctor vs patient), Permissions, Groups

Activities: Restrict admin pages, Create role-based dashboards

Task 1: Create Django view for 'My Orders' page (like Zomato) and use `@login_required` decorator to ensure only logged-in users access it.

Explanation & Approach: `@login_required` decorator redirects unauthenticated guests to login page.

```
# views.py
from django.contrib.auth.decorators import login_required
from django.shortcuts import render

@login_required(login_url='/accounts/login/')
def my_orders_view(request):
    orders = request.user.orders.all()
    return render(request, 'orders/my_orders.html', {'orders': orders})
```

Task 2: Implement custom user roles 'Seller' and 'Buyer'. Restrict access to 'Post Product' page so only 'Seller' role accesses it using permissions.

Explanation & Approach: Custom user roles enforced through user group permissions.

```
# views.py
from django.contrib.auth.decorators import login_required, permission_required
from django.shortcuts import render, HttpResponseRedirect

@login_required
def post_product_view(request):
    # Check if user has 'Seller' group
    if not request.user.groups.filter(name='Seller').exists():
        return HttpResponseRedirect("Access Denied: Only Sellers can post products!", status=403)

    return render(request, 'seller/post_product.html')
```

Task 3: Create two groups in Django admin panel: 'MovieCritic' and 'MovieFan'. Assign different permissions: MovieCritic can add/edit reviews; MovieFan only view.

Explanation & Approach: Programmatically configures permission objects for standard Django groups.

```
# Setup script (python manage.py shell)
from django.contrib.auth.models import Group, Permission
from django.contrib.contenttypes.models import ContentType
from movies.models import Review

content_type = ContentType.objects.get_for_model(Review)
```

```
# Permissions
can_add = Permission.objects.get(codename='add_review', content_type=content_type)
can_change = Permission.objects.get(codename='change_review', content_type=content_type)
can_view = Permission.objects.get(codename='view_review', content_type=content_type)

# Create Groups
critic_group, _ = Group.objects.get_or_create(name='MovieCritic')
critic_group.permissions.set([can_add, can_change, can_view])

fan_group, _ = Group.objects.get_or_create(name='MovieFan')
fan_group.permissions.set([can_view])
```

Task 4: Build role-based dashboard view: when user logs in, redirect to different dashboard based on group ('Seller' sees management; 'Buyer' sees order history).

Explanation & Approach: *Inspects group membership during login redirection logic.*

```
# views.py
from django.shortcuts import redirect

def dashboard_redirect(request):
    user = request.user
    if user.groups.filter(name='Seller').exists():
        return redirect('seller_dashboard')
    elif user.groups.filter(name='Buyer').exists():
        return redirect('buyer_dashboard')
    else:
        return redirect('default_dashboard')
```

Task 5: Restrict access to Django admin page for custom model ('Playlist') so only users in 'Admin' group can view or edit it.

Explanation & Approach: *Overrides ModelAdmin.has_module_permission to enforce group restriction in admin panel.*

```
# admin.py
from django.contrib import admin
from .models import Playlist

@admin.register(Playlist)
class PlaylistAdmin(admin.ModelAdmin):
    def has_module_permission(self, request):
        return request.user.groups.filter(name='Admin').exists() or
        request.user.is_superuser
```